

Below is a **very detailed master table of Kafka terminology** used across **all Kafka versions** (classic Kafka, ZooKeeper-based Kafka, and modern KRaft Kafka), along with **real-time use cases + examples**.

✔ **Kafka Detailed Terminology (All Versions) + Real-time Use Cases + Examples (Master Table)**

Legend

- **Kafka Core** = present from early Kafka
- **Modern Kafka** = stronger relevance after Kafka 0.10+
- **KRaft** = Kafka without ZooKeeper (Kafka 2.8+ / 3.x+)

1) Kafka Architecture / Cluster Terminology

Term	Meaning (Detailed)	Real-time Use Case	Example
Kafka Cluster	Collection of Kafka brokers working together as one system	Large scale messaging between microservices	5 brokers running in AWS across 3 AZs
Broker	Kafka server node that stores data + serves clients	Horizontal scaling + fault tolerance	broker-1, broker-2, broker-3
Node ID	Unique ID for broker/controller node	Routing, metadata, leader election	node.id=1
Controller	Node that manages partition leadership & cluster metadata	When a broker fails, controller elects new leaders	broker-2 becomes controller
Active Controller	Currently elected controller	Ensures only one controller in a cluster at a time	Active controller handles reassignments
Controller Quorum (KRaft)	Group of nodes storing metadata via Raft	Kafka without ZooKeeper	3 controller nodes
Metadata	Cluster info: topics, partitions, ISR, configs	Clients need metadata before producing/consumi	consumer fetches metadata

		ng	
Cluster ID	Unique identifier for Kafka cluster	Multi cluster setups	used for monitoring/debug

2) Kafka Storage + Topic Terminology

Term	Meaning	Use Case	Example
Topic	Logical stream/channel of messages	Segregate business domains	orders, payments, shipment
Partition	Physical shard of topic; ordered log	Scaling + parallel consumption	orders-0 ... orders-9
Partition Count	Total partitions in topic	Throughput scaling	12 partitions for high traffic
Replica	Copy of a partition stored on other brokers	Fault tolerance	replication.factor=3
Leader Replica	Broker handling reads/writes for partition	High availability	broker-1 is leader for orders-3
Follower Replica	Replica that mirrors leader	Failover	broker-2 follows broker-1
ISR (In Sync Replicas)	Replicas fully caught up	Reliability guarantee	ISR = {broker-1,2,3}
OSR	Out-of-sync replicas	Alerting + reliability risk	broker-3 lagging
Under Replicated Partition (URP)	ISR smaller than replication factor	Critical reliability warning	replication=3 but ISR=2
Log	Append-only storage for partition	Event storage & replay	order events stored sequentially
Segment	Partition log file chunk	Storage management	1GB segments

Log Segment Roll	New segment created when threshold reached	Performance	segment.roll.ms=1h
Retention	How long Kafka keeps data	Replay events for analytics	retention.ms=7days
Compaction	Keeps latest record per key	Latest state storage	user profile updates
Tombstone	Record with null value to delete key (compact topics)	Remove keys	delete user session
Log Cleanup	Background deletion/compaction	Storage control	cleanup.policy=compact
High Watermark	Offset up to which messages committed	Consumer safety	consumers read only committed
LEO (Log End Offset)	Latest produced offset	Replication lag calculation	follower LEO behind leader

3) Producer Terminology

Term	Meaning	Use Case	Example
Producer	Application writing messages	Event publishing	order service produces OrderCreated
Record / Message	The data sent to Kafka	Event payload	JSON / Avro / Protobuf
Key	Identifies message grouping; affects partitioning	Ordering per entity	key=ordered
Value	Actual payload	Business data	{orderId:101, amount:500}
Headers	Metadata in record	Tracing, versioning	traceId, schemaVersion
Partitioning	Decides which partition gets the record	Ensure ordering for key	same orderId goes same partition

Partitioner	Logic selecting partition	Custom distribution	partition by customerId
Sticky Partitioner	Optimized batching strategy	High throughput	Kafka default in newer clients
Batch	Group of records sent together	Efficiency	batch.size=32KB
Acks	Required acknowledgments	Guarantee durability	acks=all
Idempotent Producer	Prevent duplicates on retries	Exactly-once publishing	enable.idempotence=true
Retry	Automatic resend on failure	Network instability	retries=10
Delivery Timeout	Maximum time to deliver message	Producer SLA	delivery.timeout.ms
In-flight Requests	Max unacked requests	Throughput tuning	max.in.flight.requests.per.connection
Compression	Reduce network/storage size	Efficiency	gzip/snappy/zstd
Serializer	Converts object to bytes	Standard formats	StringSerializer, AvroSerializer
Producer Interceptor	Hook to modify messages	Auditing/metrics	add extra headers

4) Consumer Terminology

Term	Meaning	Use Case	Example
Consumer	App reading messages	Event handling	payment service consumes orders
Consumer Group	Multiple consumers share partitions	Scalability	3 instances of payment-service
Group Coordinator	Broker managing group membership	Rebalancing	assigns partitions
Rebalance	Partition	Autoscaling	when consumer crashes

	reassignment after join/leave		
Cooperative Rebalancing	Incremental partition moves	Min downtime	modern strategy
Poll	Consumer fetch cycle	Controls flow	consumer.poll()
Offset	Position inside partition	Track progress	offset=50213
Commit Offset	Save progress in Kafka	Recovery	enable.auto.commit=false
Auto Commit	Kafka commits offsets automatically	Simple apps	enable.auto.commit=true
Manual Commit	App commits after processing	Avoid data loss	commitSync()
Offset Reset	What to do if no offset	New consumer behavior	earliest/latest
Lag	Difference between produced and consumed	Monitoring	lag=10000
Fetch	Pull records from brokers	Throughput	fetch.min.bytes tuning
Max Poll Interval	Time allowed between polls	Long processing	max.poll.interval.ms
Heartbeat	Consumer group liveness	Detect dead consumers	heartbeat.interval.ms

5) Delivery Semantics Terminology (Critical)

Term	Meaning	Use Case	Example
At-most-once	No retries; may lose messages	Non-critical logs	metrics sampling
At-least-once	Retry possible; duplicates possible	Most business apps	order pipeline
Exactly-once	No duplicates end-to-	Banking / billing	ledger processing

(EOS)	end (Kafka transactional)		
Transactions	Atomic writes + offset commits	EOS	payment service writes + commits
Transactional ID	Identifies transactional producer	Prevent fencing issues	transactional.id=payment-tx
Fencing	Prevent old producer instances writing	Safety	avoids corruption
Isolation Level	read_committed/read_uncommitted	Consistency control	consumer uses read_committed

6) Kafka Streams Terminology

Term	Meaning	Use Case	Example
Kafka Streams	Java library for stream processing	Real-time analytics	detect fraud instantly
Topology	Graph of stream operators	Pipeline definition	map → filter → join
Processor	Processing node	Complex transformations	custom processor
KStream	Stream of events (append-only)	Event processing	clickstream events
KTable	Changelog table (latest state per key)	State storage	customer balance
GlobalKTable	Replicated lookup table	Fast joins	country codes table
State Store	Local embedded storage (RocksDB)	Aggregations/joins	store counts per product
Changelog Topic	Kafka topic backing state	Recovery	restore state after restart
Stream-Table Join	Join event stream with state	enrich events	order + customer profile

Windowing	Time-based grouping	Analytics	5-min rolling count
Grace Period	Late events allowance	Accuracy	allow late payments
Punctuation	Periodic action trigger	cleanup, emit results	every minute output stats

7) Kafka Connect Terminology

Term	Meaning	Use Case	Example
Kafka Connect	Framework for ingestion/extraction	Avoid writing custom ETL	DB → Kafka
Connector	A plugin definition (source/sink)	Integration	JDBC Source
Source Connector	Reads from external system → Kafka	CDC	MySQL → Kafka
Sink Connector	Kafka → external system	Data export	Kafka → Elasticsearch
Worker	Process running connectors	Scaling	3 connect workers
Task	Unit of parallelism under connector	Throughput	10 tasks
SMT (Single Message Transform)	Light transformations	Schema cleanup	rename fields
DLQ (Dead Letter Queue)	Stores failed records	Reliability	invalid schema events
Offset Storage	Connect tracks source offsets	Resume ingestion	MySQL binlog position

8) Schema / Serialization Terminology

Term	Meaning	Use Case	Example
Schema Registry	Stores schemas (Avro/Protobuf/JSON Schema)	Prevent breaking changes	Confluent SR

Avro	Compact binary schema-based format	High perf data	order.avsc
Protobuf	Strong typing with versioning	Microservices	order.proto
JSON Schema	Validated JSON format	Flexible APIs	events in JSON
Compatibility	Schema evolution rule	Prod safety	BACKWARD compatible

9) ZooKeeper (Legacy Kafka) Terminology

Term	Meaning	Use Case	Example
ZooKeeper	External coordination service (legacy)	Metadata storage & election	Kafka < KRaft
ZNode	ZooKeeper metadata node	cluster state	brokers ids
ZK Session	Connection to ZooKeeper	detect failures	session timeout

10) KRaft (Modern Kafka) Terminology

Term	Meaning	Use Case	Example
KRaft	Kafka Raft metadata mode	ZooKeeper-less Kafka	Kafka 3.x clusters
Raft	Consensus protocol for metadata	high reliability	leader election
Controller Log	Metadata log replicated using Raft	cluster operations	topic creation

11) Security Terminology

Term	Meaning	Use Case	Example
TLS/SSL	Encryption in transit	protect data	SSL between clients & broker

SASL	Authentication mechanism	secure login	SASL/PLAIN
SCRAM	Password hashing mechanism	auth	SASL/SCRAM-SHA-512
Kerberos	Enterprise auth	Hadoop-like infra	SASL/GSSAPI
ACL	Access control list	authorization	allow produce on orders
RBAC	Role based access	enterprise governance	admin, read-only

12) Performance / Ops / Reliability Terminology

Term	Meaning	Use Case	Example
Throughput	Messages per second	tuning	200k msg/s
Latency	Time from produce to consume	SLA	<50ms
Backpressure	Consumers can't keep up	reliability	increase partitions
Reassignment	Moving partitions between brokers	scaling	add broker-6
Rebalancing Storm	frequent rebalances	production issue	unstable consumers
Quotas	Limit produce/consume rates	fairness	per-client quotas
Metrics	Kafka stats for monitoring	alerting	lag, urp, latency

✓ Real-world Kafka Terminology Example (End-to-End)

Scenario: E-commerce system

- Topic: orders
- Partitions: 12
- Replicas: 3

- Producer: order-service sends OrderCreated
- Consumer group: payment-service consumes
- Kafka Streams: aggregates per customer spend
- Kafka Connect: exports to ElasticSearch
- Schema Registry: manages Avro schema evolution
- EOS: payment-service uses transactions to avoid duplicates

If you want, I can also create a **separate table only for configuration keywords** like:

- acks, linger.ms, batch.size, compression.type
- min.insync.replicas, unclean.leader.election.enable
- fetch.min.bytes, max.poll.records
- num.partitions, log.retention.hours

✓ and include their **recommended production values**.

Below is a **separate master table ONLY for Apache Kafka configuration keywords** (across versions), including:

- **Config key**
- **Component** (Broker/Producer/Consumer/Streams/Connect)
- **Meaning**
- **Real-time use case**
- **Recommended production value (typical)**
- **Example**

1) ✓ Kafka Broker / Server Config Keywords (server.properties)

Config Key	Component	Meaning	Real-time Use Case	Recommended Prod Value (Typical)	Example
broker.id	Broker	Unique broker identifier (legacy; KRaft uses node.id)	Identify broker in cluster	Unique integer	broker.id=1
node.id	Broker (KRaft)	Node ID for broker/controller	Required in KRaft clusters	Unique integer	node.id=1

process.roles	Broker (KRaft)	Defines node role: broker/controller	Separate controller nodes	broker or controller or both	process.roles=broker,controller
controller.quorum.voters	Broker (KRaft)	Controller quorum set	Leader election for metadata	3 or 5 voters	controller.quorum.voters=1@h1:9093,2@h2:9093,3@h3:9093
listeners	Broker	Protocol+host+port brokers bind to	Internal/external access	Define separate internal/external	listeners=INTERNAL://:9092,EXTERNAL://:9094
advertised.listeners	Broker	Address clients use to connect	Docker/K8s fixed networking	Set routable hostname	advertised.listeners=EXTERNAL://kafka1.my.com:9094
listener.security.protocol.map	Broker	Maps listener name to protocol	Multi listener security	map internal plaintext + external SSL	INTERNAL:PLAINTEXT,EXTERNAL:SSL
inter.broker.listener.name	Broker	Listener for broker-to-broker	Secure internal traffic	INTERNAL	inter.broker.listener.name=INTERNAL
num.network.threads	Broker	Network thread pool	High throughput clusters	3-8+	num.network.threads=8
num.io.threads	Broker	Disk IO processing threads	Faster disk utilization	8-32	num.io.threads=16
socket.send.buffer.bytes	Broker	TCP send buffer	Improve throughput	OS tuned	socket.send.buffer.bytes=102400
socket.receive.buffer.bytes	Broker	TCP receive buffer	Improve throughput	OS tuned	socket.receive.buffer.bytes=102400
log.dirs	Broker	Storage directories	Multi disk performance	multiple mount points	log.dirs=/kafka1,/kafka2

num.partitions	Broker	Default partitions for new topics	Default scaling	6–24 depending load	num.partitions=12
default.replication.factor	Broker	Default replication factor	HA by default	3	default.replication.factor=3
min.insync.replicas	Broker/Topic	Minimum ISR required when acks=all	Avoid losing committed data	2 (if RF=3)	min.insync.replicas=2
unclean.leader.election.enable	Broker	Allow out-of-sync replica as leader	Prevent data loss	false	unclean.leader.election.enable=false
auto.create.topics.enable	Broker	Auto-create topics on first use	Avoid accidental topics	false	auto.create.topics.enable=false
delete.topic.enable	Broker	Allows deleting topics	Cleanup in non-prod/prod controlled	true but restricted via ACL	delete.topic.enable=true
log.retention.hours	Broker	Retention duration (time-based)	Storage control	72–168	log.retention.hours=168
log.retention.bytes	Broker	Retention size limit	Cap storage per partition	depends	log.retention.bytes=1073741824
log.segment.bytes	Broker	Segment file size	Faster compaction & cleanup	512MB–1GB	log.segment.bytes=1073741824
log.cleanup.policy	Broker/Topic	delete / compact	compact for state topics	delete or compact	log.cleanup.policy=delete
log.cleaner.enable	Broker	Enable	compaction	true	log.cleaner.enable

nable		compaction cleaner threads	n topics		=true
log.cleaner.threads	Broker	Threads for log cleaning	Compaction performance	2-4	log.cleaner.threads=2
message.max.bytes	Broker	Max record size broker accepts	Large messages allowed	1MB-10MB	message.max.bytes=2097152
replica.fetch.max.bytes	Broker	Max bytes replica fetches	Large messages replication	match message sizes	replica.fetch.max.bytes=2097152
replica.lag.time.max.ms	Broker	Replica considered dead after lag	Leader election stability	10s-30s	replica.lag.time.max.ms=30000
controlled.shutdown.enable	Broker	Graceful shutdown	rolling deployments	true	controlled.shutdown.enable=true

2) ✓ Kafka Topic-Level Config Keywords (per-topic configs)

These override broker defaults.

Config Key	Meaning	Use Case	Typical Prod Value	Example
retention.ms	Retention time for topic	Short-lived event topics	1-7 days	retention.ms=604800000
retention.bytes	Max retention size	Storage cap	org-specific	retention.bytes=2147483648
cleanup.policy	delete / compact	compact for user state	delete or compact	cleanup.policy=compact
segment.bytes	Segment size	Compaction tuning	512MB-1GB	segment.bytes=536870912

segment.ms	Time-based segment rolling	Time partitioning	1h-24h	segment.ms=3600000
min.insync.replicas	ISR requirement	prevent data loss	2 (RF=3)	min.insync.replicas=2
compression.type	Broker-side compression	reduce storage	producer	compression.type=producer
max.message.bytes	Topic max msg size	allow big payload	1-10MB	max.message.bytes=2097152

3) ✓ Kafka Producer Config Keywords (producer.properties / Java producer configs)

Config Key	Component	Meaning	Real-time Use Case	Recommended Prod Value	Example
bootstrap.servers	Producer	Broker list for initial connection	connect to cluster	comma list	k1:9092,k2:9092
key.serializer	Producer	Serialize record key	ordering by key	String/Avro	StringSerializer
value.serializer	Producer	Serialize record value	payload conversion	Avro/JSON/Protobuf	KafkaAvroSerializer
acks	Producer	Required ack level	durability	all	acks=all
enable.idempotence	Producer	Prevent duplicates on retry	financial events	true	enable.idempotence=true
retries	Producer	Retry send	transient network issues	high / default	retries=2147483647
max.in.flight.requests.per.connection	Producer	In-flight batch limit	preserve order with EOS	5 or 1	max.in.flight...=5
delivery.timeout.ms	Producer	Total max delivery time	SLA control	120000	delivery.timeout.ms=120000

request.time out.ms	Producer	Wait time for request	slow broker handling	30000	request.timeout.ms=30000
linger.ms	Producer	Wait time to batch more records	throughput improvement	5-50	linger.ms=20
batch.size	Producer	Batch size per partition	high throughput	32KB-128KB	batch.size=65536
buffer.memo ry	Producer	Total memory for buffering	burst handling	64MB-512MB	buffer.memory=134217728
compression.type	Producer	compress message payload	reduce network	zstd / snappy	compression.type=zstd
max.request.size	Producer	Max request size	large messages	match broker/topic	max.request.size=2097152
client.id	Producer	Client identifier	tracing/metrics	meaningful name	client.id=order-service

4) ✓ Kafka Consumer Config Keywords (consumer.properties / Java consumer configs)

Config Key	Component	Meaning	Use Case	Recommended Prod Value	Example
bootstrap.servers	Consumer	Broker connection points	join cluster	list	k1:9092,k2:9092
group.id	Consumer	Consumer group name	parallel consumption	service name	group.id=payment-service
enable.auto.commit	Consumer	Auto commits offsets	simple consumers	false in prod	enable.auto.commit=false

auto.commit.interval.ms	Consumer	Auto commit interval	low-critical consumption	5000	auto.commit.interval.ms=5000
auto.offset.reset	Consumer	earliest/latest/none	new group behavior	earliest (often)	auto.offset.reset=earliest
key.deserializer	Consumer	Deserialize key	key usage	StringDeserializer	StringDeserializer
value.deserializer	Consumer	Deserialize value	read payload	Avro/JSON	KafkaAvroDeserializer
max.poll.records	Consumer	Max records per poll	batch processing	100-1000	max.poll.records=500
fetch.min.bytes	Consumer	Minimum fetch size	throughput tuning	1KB-1MB	fetch.min.bytes=1048576
fetch.max.wait.ms	Consumer	Wait to accumulate fetch	throughput	100-500	fetch.max.wait.ms=500
max.poll.interval.ms	Consumer	Max time between polls	long processing	5-30 minutes	max.poll.interval.ms=600000
session.timeout.ms	Consumer	Failure detection timeout	rebalance speed	10-45 sec	session.timeout.ms=30000
heartbeat.interval.ms	Consumer	heartbeat frequency	keep membership	1/3 of session	heartbeat.interval.ms=10000
isolation.level	Consumer	transactional read mode	EOS safety	read_committed	isolation.level=read_committed

5) ✓ Kafka Streams Config Keywords

Config Key	Meaning	Real-time Use Case	Recommended Value	Example
application.id	Streams app identity	state stores, changelog topics	unique per app	application.id=fraud-checker

bootstrap.servers	Cluster bootstrap	connect	list	k1:9092
processing.guarantee	at_least_once/exactly_once_v2	banking/fraud	exactly_once_v2	processing.guarantee=exactly_once_v2
state.dir	Local state path	RocksDB storage	SSD path	state.dir=/kafka-streams
commit.interval.ms	Commit frequency	performance vs safety	1000-10000	commit.interval.ms=1000
cache.max.bytes.buffering	state caching	reduce writes	default/ tuned	cache.max.bytes.buffering=10485760

6) ✓ Kafka Connect Config Keywords

Config Key	Meaning	Use Case	Recommended	Example
bootstrap.servers	Connect cluster	connect to Kafka	list	k1:9092
group.id	Connect worker group	distributed mode	connect-cluster	group.id=connect-cluster
key.converter	Convert record keys	JSON/Avro keys	Avro in prod	AvroConverter
value.converter	Convert record values	schema support	Avro	AvroConverter
config.storage.topic	connector configs topic	durability	RF=3	connect-configs
offset.storage.topic	offsets topic	resume ingestion	RF=3	connect-offsets
status.storage.topic	status tracking topic	monitor tasks	RF=3	connect-status
errors.tolerance	skip errors?	robustness	all for resilience	errors.tolerance=all

errors.deadletterqueue.topic.name	DLQ topic	capture failed records	enabled	connect-dlq

✔ Want an “All-in-One Kafka Config Master Table PDF/Excel”?

If you want, I can generate:

1. **Kafka configuration cheat sheet (300+ configs)**
2. grouped by:
 - performance
 - durability
 - partitions/replication
 - consumer lag tuning
 - producer batching/compression
 - security configs (SSL/SASL)
 - KRaft configs

Just tell me: **PDF or Excel?**